

Deep Learning: Training DNNs – Part II

1. Data Preprocessing

Why Preprocess Data?

Raw data collected from the real world is rarely clean or ready to use. It usually has various problems — missing values, inconsistencies, noise, and features that are on very different numerical scales. Feeding such data directly into a neural network leads to poor and unpredictable training. Preprocessing fixes this.

There are four main reasons we preprocess data before training:

- 1. Data Quality:** Missing values, noise, and corrupted entries must be handled before training. A model trained on noisy or incomplete data will learn the wrong patterns.
- 2. Algorithm Requirements:** Gradient descent based models are sensitive to the format and scale of the data. Features with large numerical ranges (e.g. salary in lakhs vs age in years) can dominate the gradient updates and make training unstable.
- 3. Better Model Performance:** Removing irrelevant features and reducing the influence of outliers helps the model focus on what actually matters and generalize better to new data.
- 4. Efficiency:** Techniques like dimensionality reduction compress the data while retaining useful information, making training faster.

How to Preprocess Data?

The general goal is to make data 'scaled, stationary, and representative.' Here are the key techniques:

Scaling / Standardization / Normalization

The most common preprocessing step is standardization — subtracting the mean and dividing by the standard deviation so that every feature has zero mean and unit variance. This is **called z-score normalization**:

$$z = (x - \mu) / \sigma$$

💡 μ is the mean of the feature, σ is the standard deviation. After this, the feature has mean 0 and std 1.

Other normalization methods also exist, such as Min-Max normalization which scales everything to the [0, 1] range. The right choice depends on the problem.

Categorical Encoding

Categorical variables (like city names or product types) need to be converted into numbers. The simplest way is **One-Hot Encoding** — creating a separate binary column for each category.

One-hot encoding works well for **low-cardinality categories** (few unique values). For **high-cardinality categories** (thousands of values, like words in NLP), one-hot becomes impractical. Instead, an **embedding layer** is learned as part of training it maps each category to a dense, low-dimensional vector that captures semantic meaning. This is what word embeddings in NLP do.

Task-Specific Preprocessing

Some preprocessing choices depend heavily on the domain.

In Computer Vision, a common step is **zero-centering the pixel values** — **subtracting the mean image so that the data is centered around zero**.

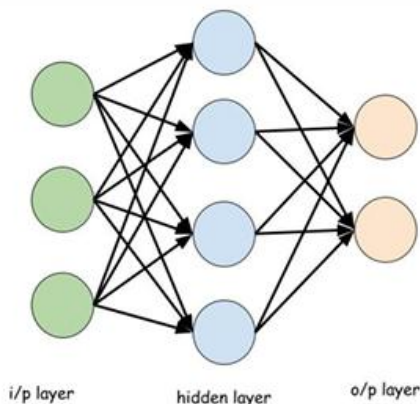
In NLP, **tokenization** is a key step — breaking text into tokens (words or sub-words) in a way that preserves as much of the original context as possible.

2. Weight Initialization

Why Does Initialization Matter?

Some optimization algorithms are not iterative — they find their answer in one shot and don't care about starting conditions. But **deep neural networks are trained iteratively** (via gradient descent), so the **starting values of the weights** determine whether **training will converge at all, and how quickly**.

Modern initialization strategies are **mostly simple heuristics**. This is because deep network optimization is not yet fully understood mathematically. These strategies aim to achieve 'nice' properties at the start — though we cannot always guarantee those properties persist throughout training.

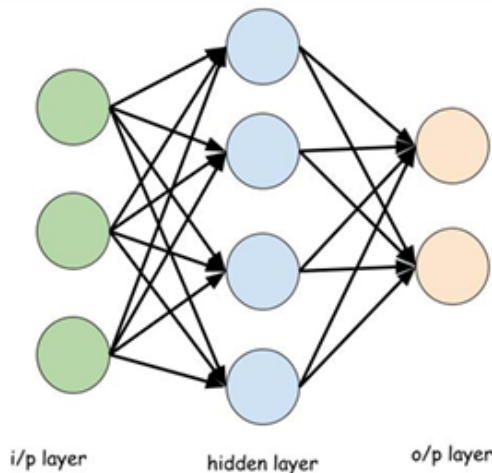


- What if all the parameters are initialized to zero?
- Or, a different constant?
- Leads to a failure mode (often known as the '**symmetry**' problem)
- Hence, we need different values as weights!

The Symmetry Problem: Why Not Initialize to Zero?

If all the weights are initialized to zero (or any single constant), **every neuron in a layer computes the exact same output during the forward pass**. They also receive the **exact same gradient during backprop**. This means they **all update identically** — the network never learns to differentiate features. **This failure is called the symmetry problem.**

Initializing all weights to the same constant, even a non-zero constant, causes the same issue. We need different layer weights so that different neurons learn different things.



- Is it good enough to have different parameters?
- Large weights → **exploding gradients**
- Small ones → **vanishing gradients**
- Different weights → different o/p range of the neurons

Why Not Just Use Large or Small Random Values?

Random values break symmetry, but the scale matters enormously:

Too large: Gradients explode during backpropagation — the updates become so large the training diverges.

Too small: Gradients vanish — the signal fades away as it passes back through layers, and the early layers learn nothing. For example, a 6-layer MLP with tanh activations initialized with very small weights will have activations shrinking layer by layer toward zero. All gradients become zero and learning stops completely.

Initializing with small random numbers looks like this:

```
W = 0.001 * np.random.randn(dl, dl-1)
```

💡 This may be okay for shallow networks, but fails for deep ones due to the vanishing activation problem.

2a. Xavier / Glorot Initialization (2010)

Xavier Glorot and Yoshua Bengio (2010) proposed a principled way to choose the initial weight scale.

- **The core idea: keep the variance of activations the same across all layers during the forward pass, and**
- **keep the variance of gradients the same across all layers during the backward pass.**
- **If both conditions hold, signals flow cleanly in both directions without vanishing or exploding.**

Derivation — Forward Pass

- Let's look at the forward pass
- Consider a single layer, $y = Wx$, i.e. $y_i = \sum_{j=1}^{d_{l-1}} x_j \cdot w_j$
- $\text{var}(y_i) = d_{l-1} \cdot \text{var}(x_i \cdot w_i)$ (Assuming w_i and x_i are i.i.d)
- $\text{var}(y_i) = d_{l-1} \cdot \left(E(x_i^2) \cdot E(w_i^2) - E(x_i)^2 \cdot E(w_i)^2 \right)$ (Assuming x and w are independent and zero mean)
- $\text{var}(y_i) = d_{l-1} \cdot \text{var}(x_i) \cdot \text{var}(w_i)$ Assuming $(x_i$ and w_i are zero-mean)
- $\rightarrow \text{var}(w_i) = \frac{1}{d_{l-1}}$

Backward Pass Constraint

- Similar logic applies to the gradients moving backward during the backprop forward pass
- $\rightarrow \text{var}(w_i) = \frac{1}{d_l}$
- Xavier and Bengio proposed using the average of these two constraints as a compromise $\text{var}(w_i) = \frac{2}{d_l + d_{l-1}}$
- If we use Uniform distribution $U(-a, a)$, $a = \sqrt{\frac{6}{d_l + d_{l-1}}}$

2b. Kaiming He / MSRA Initialization (for ReLU networks)

Xavier initialization was designed assuming linear activations (or activations like tanh that are approximately linear near zero). ReLU activations are different — they zero out half the neurons on average. This effectively halves the variance at each layer, which would cause vanishing activations in deep ReLU networks.

Kaiming He et al. (Microsoft Research Asia, MSRA) proposed a fix: account for the fact that ReLU zeroes out half the inputs by doubling the variance:

```
std = sqrt( 2 / d_{l-1} )
```

In code:

```
W = np.random.randn(d_l, d_{l-1}) * np.sqrt(2 / d_{l-1})
```

💡 The factor of 2 in the numerator compensates for the half-zeroing effect of ReLU.

2c. Initialization for Residual Networks (2015)

Residual networks (ResNets) have skip connections — a layer's output is $F(x) + x$ (the residual plus the identity). This causes a variance problem at initialization:

$$\text{Var}(F(x) + x) > \text{Var}(x)$$

💡 The variance grows because you are adding two independent sources together.

If we apply MSRA initialization to residual networks, the variance grows at every residual block — leading to exploding activations.

The fix: initialize the scale parameter γ of the last Batch Normalization layer in each residual branch to zero. This makes the entire residual branch output zero at initialization:

$$F(x) = 0 \quad (\text{initially})$$

$$\text{Var}(x + F(x)) = \text{Var}(x + 0) = \text{Var}(x)$$

💡 By making the residual branch output nothing at the start, the skip connection carries the input cleanly and variance is preserved.



- MSRA initialization:
 $\text{Var}(F(x)+x) > \text{Var}(x)$
- Variance grows!
- Solution: Initialize the scale parameter (γ) of the last BN layer with zero
- The entire residual branch outputs zero (initially!) $\rightarrow$
 $\text{Var}(x+F(x)) = \text{Var}(x)$

Modern LLMs: Truncated Normal Initialization

Most modern Large Language Models (GPT, etc.) don't strictly follow Xavier or He initialization. Instead, they use a Truncated Normal Distribution with fixed hyperparameters:

$$\mu = 0 \text{ and } \sigma = 0.02$$

This is an empirical heuristic — the small $\sigma = 0.02$ keeps weights small enough to avoid activation saturation from the start. For the output projection layers inside residual

blocks, the standard deviation is further scaled down by the depth of the network to prevent variance growth:

$$\sigma_{\text{res}} = \frac{0.02}{\sqrt{(2.N)}}$$

💡 *N is the number of residual layers. As the network gets deeper, the output projection weights start smaller.*

3. Regularization in Deep Networks

The Core Idea

Regularization refers to any technique that prevents a model from overfitting — that is, memorizing the training data rather than learning general patterns. Regularization methods work by introducing a trade-off: they intentionally increase the model's bias slightly in exchange for a significant reduction in variance. The net result is better generalization to new data.

This trade-off only helps if it's profitable — if the variance reduction is larger than the bias increase. The empirical finding in deep learning is that the best model is usually a large, overparameterized model that has been appropriately regularized, not a small model that is just barely big enough.

Regularization Techniques Overview

Deep learning uses many forms of regularization:

- Parameter Norm Penalties (L2 and L1)
- Dataset Augmentation
- Noise Robustness
- Semi-Supervised Learning
- Multi-Task Learning (Parameter Sharing)
- Sparse Representation
- Dropout
- Batch Normalization

3a. Parameter Norm Penalties

The simplest form of regularization adds a penalty term to the loss function that discourages the weights from becoming too large. In neural networks, we typically

regularize only the weights of the affine (linear) transformations, not the biases. Why not biases? Because a bias only controls a single variable, whereas a weight connects two neurons and has much more influence. Regularizing biases can cause underfitting.

- ① L_2 parameter regularization: $\tilde{\mathcal{J}} = \frac{\alpha}{2} w^T w + \mathcal{J}(w; X, y)$
- ② L_1 regularization: $\tilde{\mathcal{J}} = \alpha |w|_1 + \mathcal{J}(w; X, y)$
- ③ Norm penalties induce different desired behaviors based on the exact penalty imposed

L2 Regularization (Weight Decay)

L2 adds a penalty proportional to the sum of squared weights:

💡 α (alpha) is the regularization strength. $w^T w$ is the squared L2 norm of the weights. This penalizes large weights and pushes them toward zero — the optimizer must balance fitting the data AND keeping weights small.

The effect: L2 encourages all weights to be small but non-zero. It is also called weight decay because during gradient updates, the weights are literally decayed by a factor at each step.

L1 Regularization

L1 adds a penalty proportional to the sum of absolute values of the weights:

💡 $|w|_1$ is the L1 norm — the sum of absolute values. α controls the strength of regularization.

The key difference from L2: L1 regularization tends to produce sparse solutions — it drives many weights to exactly zero, effectively removing those connections from the network. This makes L1 useful when you want automatic feature selection.

3b. Dataset Augmentation

The single best way to make a model generalize better is to train it on more data. More diverse examples force the model to learn truly general patterns rather than memorizing specific training samples. In practice, however, collecting more labeled data is expensive and time-consuming.

Dataset Augmentation solves this by creating artificial, modified versions of existing training examples and adding them to the training set. These are sometimes called fake data, but they are physically plausible variations of real data.

Augmentation is easier for classification tasks than for density estimation (where you would need to know the exact data distribution to generate valid fake samples).

For image classification (object recognition), augmentation has been particularly effective. Common image augmentations include:

- Translating the image by a few pixels in any direction
- Rotating slightly (a few degrees)
- Flipping horizontally or vertically
- Adding mild Gaussian noise to pixel values
- Changing brightness, contrast, or color slightly

These operations greatly improve generalization. Hand-designed domain-specific augmentations can give dramatic improvements in some fields. One critical constraint: augmentations must be label-preserving. A car rotated by 5 degrees is still a car, so the label stays the same. But flipping a digit '6' upside down produces a '9' — not label-preserving.

3c. Multi-Task Learning (Parameter Sharing)

Multi-Task Learning (MTL) improves generalization by training a model on multiple related tasks simultaneously instead of just one. The model has shared layers that learn representations useful for all tasks, and task-specific layers on top.

The regularization effect comes from pressure: the shared layers are now forced to produce representations that work for multiple tasks, not just one. This is similar to having more data — multi-task samples demand that the shared parameters be more general and robust. A representation that helps with many tasks is less likely to be overfitted to any single one.

3d. Dropout (Srivastava et al., 2014)

Dropout is one of the most influential deep regularization techniques. During the forward pass of training, some neurons are randomly 'zeroed out' — completely removed from the network for that particular example. Dropped neurons contribute nothing to the output and receive no gradient during backprop.

How Dropout Works

We specify which layers to apply dropout to and a drop probability p (i.e. each neuron is dropped with probability p , retained with probability $1-p$). For each training example:

- For each neuron, independently sample a Bernoulli random variable (1 = keep, 0 = drop).
- Zero out the activations of dropped neurons.
- Run forward and backward pass only through the remaining active neurons.
- The next example uses a new, different random mask.

Why Dropout Works — Two Interpretations

Interpretation 1: Independence. Since any neuron might be dropped at any time, neurons cannot rely on any specific other neuron to always be present. This prevents co-adaptation — neurons must learn to be independently useful. The result is that the learned representation is distributed across many neurons, not concentrated in a few.

Interpretation 2: Ensemble. Each training example uses a different random mask, effectively training a different sub-network. A dense layer with n neurons has 2^n possible masks — a dense layer with just 10 neurons has 1024 possible sub-networks! Dropout trains an enormous ensemble of models that all share the same weights. This is a powerful form of model averaging.

Dropout at Test Time

At test time we don't want random behavior — we want a single deterministic prediction. The right thing to do is take the expected output over all possible masks. This is approximated by:

- ① Which model from the ensemble to use?
 $y = f(x, w, m)$ (m is the chosen binary mask)
- ② How about taking the opinion of all the experts? → 'average out' and make the o/p deterministic
- ③ $y = \mathbb{E}_m[f(x, w, m)] = \sum_m p(m) \cdot f(x, w, m)$
- ④ Leads to dropping no unit but multiply the activations with the probability of retaining

💡 m is a binary mask. We take a weighted average of the outputs of all possible sub-networks.

In practice this is done by a simple trick — instead of dropping neurons, keep all of them but multiply each activation by the probability of retaining it ($1 - p$). This ensures the expected output at test time matches what was seen during training. This is called weight scaling or inverted dropout.

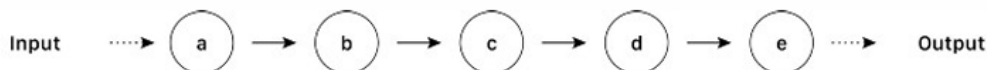
Where to Apply Dropout?

Dropout is mainly applied to fully-connected (dense) layers because they have the most parameters and are most prone to overfitting. Convolutional layers have weight sharing and far fewer parameters, so dropout is less commonly applied there. Important note: with the rise of ResNets and modern architectures that use Batch Normalization, dropout has become less commonly used — BN provides its own regularization effect.

3e. Batch Normalization (BN)

- ① Gradient Descent converges faster with feature scaling ($x \leftarrow \frac{x-\mu}{\sigma}$)
- ② Batch Normalization (BN) is a normalization method for intermediate layers of NNs → performs whitening to the intermediate layer activations
- ① Originally introduced to handle the internal covariate shift (ICS)
- ② BN makes the activation of each neuron to be Gaussian distributed
- ③ ICS is undesirable because the layers need to adapt to the new distribution of activations
- ④ With BN, it is reduced to new pair of parameters, but the distribution remains Gaussian

- ① Mitigates interdependency between hidden layers during training



- ② $\partial(a) = \partial(b) \cdot \partial(c) \cdot \partial(d) \cdot \partial(e)$
- ③ if we want to adjust the input distribution of a specific hidden unit, we need to consider the whole sequence of layers (w/o BN)
- ④ BN acts like a valve which holds back the flow, and allows its regulation using β and γ

Benefits of Batch Normalization

- 1. Faster training:** Reduces ICS, so each layer doesn't waste time chasing a shifting input distribution.
- 2. Acts as a regularizer:** The mean and variance are computed per batch, introducing slight noise into the activations. This has a regularizing effect, reducing the need for Dropout.
- 3. Allows higher learning rates:** Because BN prevents activations from exploding or vanishing, you can safely use larger learning rates without the training diverging.

4. Closing Thought: Regularization as Inductive Bias

The general principle behind all regularization is making assumptions explicit. Every regularization technique encodes some prior belief about what a good model looks like:

- L2 regularization says: 'weights should be small'
- L1 regularization says: 'most weights should be exactly zero'
- Dropout says: 'no single neuron should be critical; the representation should be distributed'
- Augmentation says: 'the label should not change under small transformations'
- Batch Normalization says: 'activations within a layer should be approximately Gaussian'

These assumptions, called inductive biases, guide the model toward solutions that are more likely to generalize well. The art of deep learning is choosing the right set of inductive biases for your problem.